CHATKONECT

All modules. All features.

A detailed module-by-module catalogue for the app owner and development team.

SOURCE EDITION / 20 OCTOBER 2026

49	900	205
MODULES	DETAILED ENTRIES	REST OPERATIONS

Each module begins with its name and access context, followed by individually numbered features, small controls and important behavior rules.

Includes a clickable module index, source references, API/event appendices and clearly marked implementation limits.

FIND A MODULE

Module index

Click a module name to open its feature list. The PDF also includes bookmarks.

01	Account signup and onboarding	4
02	Login and password recovery	5
03	App lock and two-step PIN	6
04	Sessions and signed-in devices	7
05	Profile and account data	8
06	Contacts and contact requests	9
07	QR codes and invite links	10
08	Chat list and conversation organization	11
09	Locked chats, clearing and disappearing messages	12
10	Message composer and everyday sending	13
11	Message history and reading experience	14
12	Message actions, reactions and forwarding	15
13	Starred messages	16
14	Temporary pinned messages	17
15	Typing, presence and delivery receipts	18
16	Photos, videos, documents and upload limits	19
17	Camera, voice notes and round video notes	20
18	View-once media	21
19	Chat polls	22
20	Scheduled messages	23
21	Location and live location	24
22	Groups, group members and group roles	25
23	Communities and announcement groups	26
24	Broadcast lists	27
25	Search and jump to message	28

FIND A MODULE

Module index / continued

Click a module name to open its feature list. The PDF also includes bookmarks.

26	Status and 24-hour stories	29
27	Direct and group audio/video calls	30
28	Call history and network relay support	31
29	Meeting scheduling and invitations	32
30	Meeting join, password and device check	33
31	Live meeting room and media controls	34
32	Meeting chat, reactions, hands, polls and Q&A	35
33	Host controls, breakouts, notes and attendance	36
34	Notifications, sounds and Web Push	37
35	Privacy, blocking and reporting	38
36	Presence preferences and appearance	39
37	Workspace setup, members and roles	40
38	Shared customer inbox	41
39	Team tasks and workspace activity	42
40	Limited guests and guest portal	43
41	Business storefront and automatic replies	44
42	Product catalog and product sharing	45
43	Conversation labels and quick replies	46
44	Platform apps and tenant administration	47
45	Developer integration, hosted embed and SDKs	48
46	API keys, public REST API and incoming webhooks	49
47	Admin dashboard, moderation and account control	50
48	Application shell, responsive UI and installation assets	51
49	Backend operation, data efficiency and safeguards	52

MODULE 01 / 18 ENTRIES

Account signup and onboarding

Public signup; email delivery depends on configured provider

1.01	Create an individual Personal account.
1.02	Create a new team workspace during signup.
1.03	Join an existing team using its invite code or invite link.
1.04	Join selected team groups through a limited guest signup link.
1.05	Enter name, email, phone number, password and password confirmation.
1.06	Check duplicate email and phone registrations.
1.07	Normalize phone numbers for account matching.
1.08	Automatically generate a unique username from the email address.
1.09	Accept a valid explicit username from compatible API clients.
1.10	Upload or supply an optional signup avatar; use a fallback avatar otherwise.
1.11	Send a signup email verification code when verification is enabled.
1.12	Verify the email code before submitting signup.
1.13	Resend an email verification code.
1.14	Expire verification codes and limit incorrect verification attempts.
1.15	Issue a time-limited proof after successful email verification.
1.16	Reject invalid workspace invitations before creating the account.
1.17	Create a signed-in session after successful signup.
1.18	Retain compatibility with the account OTP verification and resend endpoints.

SOURCE REFERENCES

server/controllers/authController.js
client/src/pages/auth/Signup.jsx

MODULE 02 / 17 ENTRIES

Login and password recovery

Account holders; email recovery requires delivery configuration

2.01 Sign in using email and password.

2.02 Sign in using username and password.

2.03 Sign in using phone number and password, including normalized number matching.

2.04 Show or hide password text in authentication forms.

2.05 Reject incorrect credentials with a readable response.

2.06 Reject suspended or banned accounts.

2.07 Require verified accounts when email verification is enabled.

2.08 Request a forgotten-password recovery link.

2.09 Return a generic recovery acknowledgement to protect account existence.

2.10 Reset the password using an expiring recovery token.

2.11 Change the current password from account settings.

2.12 Invalidate old tokens and sessions after password changes or resets.

2.13 Refresh an expired access session using the refresh cookie.

2.14 Rotate refresh credentials rather than reusing the old refresh token.

2.15 Log out of the current session.

2.16 Restore the signed-in account on application reload.

2.17 Apply authentication rate limits and security event logging.

SOURCE REFERENCES

server/controllers/authController.js
client/src/pages/auth/Login.jsx
client/src/pages/auth/ForgotPassword.jsx
client/src/pages/auth/ResetPassword.jsx

MODULE 03 / 11 ENTRIES

App lock and two-step PIN

Signed-in account; account lock is distinct from a chat lock

3.01 Enable a two-step application-lock PIN.

3.02 Store the PIN as a password hash rather than readable text.

3.03 Show an application lock screen when PIN verification is required.

3.04 Verify the PIN to unlock the application.

3.05 Change an existing PIN.

3.06 Disable the PIN after the required verification.

3.07 Request an email code for forgotten-PIN recovery.

3.08 Reset the PIN using a valid recovery code.

3.09 Expire recovery codes and restrict incorrect recovery attempts.

3.10 Keep PIN and PIN-recovery secrets out of user API responses.

3.11 Use the same account PIN for access to the locked-chat list.

SOURCE REFERENCES

client/src/components/LockScreen.jsx
client/src/components/PinResetForm.jsx
server/controllers/authController.js

MODULE 04 / 12 ENTRIES

Sessions and signed-in devices

Signed-in account; own sessions only

4.01 List tracked signed-in sessions and devices.

4.02 Show the device/browser label derived from its user agent.

4.03 Track session creation and recent activity.

4.04 Identify the current session in device management.

4.05 Revoke an individual session.

4.06 Revoke all other sessions while preserving the current one.

4.07 Reject tokens belonging to a revoked session.

4.08 Expire sessions after their absolute lifetime.

4.09 Expire sessions after the configured idle period.

4.10 Keep refresh token hashes instead of raw refresh tokens in storage.

4.11 Periodically update activity without writing on every request.

4.12 Recheck account status and token version on authenticated requests.

SOURCE REFERENCES

server/utils/session.js
server/controllers/authController.js
client/src/pages/SettingsPage.jsx

MODULE 05 / 12 ENTRIES

Profile and account data

Signed-in account; another user receives privacy-filtered data

5.01	View your name, username, email, phone, avatar and about text.
5.02	Edit the display name.
5.03	Edit the username with uniqueness and format checks.
5.04	Edit the phone number with uniqueness validation.
5.05	Edit the about/bio text.
5.06	Change the profile photo.
5.07	Show fallback initials/avatar when a photo is absent or hidden.
5.08	Open another person's contact/profile information.
5.09	Export account data through the authenticated export endpoint.
5.10	Delete your own account using the account danger-zone flow.
5.11	Clean associated relationships and account-owned data during deletion.
5.12	Keep password hashes, OTPs, reset tokens and PIN secrets out of safe profiles.

SOURCE REFERENCES

server/controllers/userController.js
client/src/components/modals/ModalHost.jsx
client/src/pages/SettingsPage.jsx

MODULE 06 / 21 ENTRIES

Contacts and contact requests

Signed-in users; consent-based mutual contacts

6.01	Search for a person by exact email.
6.02	Search for a person by exact username.
6.03	Search for a person by normalized phone number.
6.04	Search partial names/usernames within the permitted team directory.
6.05	View the existing contacts list and contact count.
6.06	Search/filter your displayed contacts.
6.07	Send a contact request.
6.08	View incoming contact requests.
6.09	View outgoing contact requests.
6.10	Accept an incoming request.
6.11	Reject an incoming request.
6.12	Automatically accept crossed pending requests from both people.
6.13	Prevent duplicate pending requests.
6.14	Allow a fresh request after a previous request was rejected.
6.15	Update both contact lists when a request is accepted.
6.16	Remove a contact.
6.17	Add or remove a contact from favorites.
6.18	Start a direct chat with a permitted mutual contact.
6.19	Start audio or video calls from contact actions.
6.20	Update requests and accepted contacts through realtime events.
6.21	Restrict exposure of another person's email and phone to permitted contacts.

SOURCE REFERENCES

server/controllers/contactController.js
server/controllers/userController.js
client/src/pages/ContactsPage.jsx

MODULE 07 / 13 ENTRIES

QR codes and invite links

Signed-in users; browser camera permissions for scanning

7.01	Display your contact QR code.
7.02	Share or copy the associated contact link.
7.03	Display a group invite link and QR code.
7.04	Display a workspace invite link and QR code where permitted.
7.05	Scan supported invitation QR codes with a camera.
7.06	Paste an invitation link manually when camera scanning is unavailable.
7.07	Switch available scan cameras where the scanner exposes the option.
7.08	Stop scanning and release the camera when the scanner closes.
7.09	Handle denied camera permission and unreadable QR data.
7.10	Parse supported contact, group and workspace invite formats.
7.11	Open a scanned or pasted invitation through the matching join flow.
7.12	Preserve invitation context through sign-in or signup.
7.13	Reject unsupported or malformed invite links.

SOURCE REFERENCES

client/src/components/InviteQrModal.jsx
client/src/components/ScanQrModal.jsx
client/src/components/QrScanner.jsx
client/src/lib/invite.js
client/src/pages/JoinInvite.jsx

MODULE 08 / 21 ENTRIES

Chat list and conversation organization

Signed-in users; list is personal to the account

8.01	List direct conversations and group conversations.
8.02	Show each conversation's name and avatar.
8.03	Show last-message preview and last activity time.
8.04	Show unread-message counts and unread badges.
8.05	Show sent/delivered/read state in conversation previews.
8.06	Show typing and presence hints where available.
8.07	Filter All conversations.
8.08	Filter only Unread conversations.
8.09	Filter only Groups.
8.10	Filter Archived conversations.
8.11	Search the chat list.
8.12	Open the new-chat contact picker.
8.13	Create or reuse a direct chat with the same contact.
8.14	Pin or unpin a chat in your own list.
8.15	Archive or unarchive a chat.
8.16	Mute or unmute a chat.
8.17	View the archived/filtered conversation views exposed by the sidebar.
8.18	Open broadcast lists from the chat navigation.
8.19	Select a conversation and show its header, history and composer.
8.20	Refresh chat ordering and previews after incoming messages.
8.21	Keep per-user pin/archive/mute settings separate from other participants.

SOURCE REFERENCES

client/src/components/chat/ChatSidebar.jsx
server/controllers/chatController.js
server/controllers/userController.js

MODULE 09 / 14 ENTRIES

Locked chats, clearing and disappearing messages

Chat members; PIN needed for locked-chat access

9.01 Lock a selected chat using the account PIN flow.

9.02 Hide locked chats from the ordinary conversation list.

9.03 Reveal the locked-chat list after PIN verification.

9.04 Unlock a previously locked chat.

9.05 Exclude locked conversations from global search.

9.06 Clear message history for yourself.

9.07 Remove a direct conversation from your personal view.

9.08 Leave a group through the group chat removal flow.

9.09 Reject chat removal by a nonparticipant before any write.

9.10 Enable disappearing messages for a chat.

9.11 Choose disappearing-message periods of 24 hours, 7 days or 90 days.

9.12 Turn disappearing messages off.

9.13 Apply expiry timestamps to messages sent while disappearance is enabled.

9.14 Keep personal deletions from reappearing through search/context lookup.

SOURCE REFERENCES

server/controllers/chatController.js
client/src/components/chat/ChatHeader.jsx
client/src/components/chat/RightPanel.jsx

MODULE 10 / 20 ENTRIES

Message composer and everyday sending

Chat members with posting permission

10.01	Write and send a text message.
10.02	Use multi-line message input.
10.03	Send with Enter when Enter-to-send is enabled.
10.04	Use a separate newline gesture instead of sending.
10.05	Open and close the emoji picker.
10.06	Insert an emoji into the current message.
10.07	Open the attachment/action picker.
10.08	Retain separate unsent drafts for different conversations.
10.09	Restore a saved draft after returning to a conversation.
10.10	Continue safely if browser storage is unavailable.
10.11	Show an optimistic outgoing message while the request is pending.
10.12	Replace the temporary message with the saved server message.
10.13	Show failed-send status when sending fails.
10.14	Show the reply target above the composer.
10.15	Cancel the current reply before sending.
10.16	Prevent empty text-only messages.
10.17	Validate message text length; server limit is 10,000 characters.
10.18	Use team quick replies from the composer when available.
10.19	Send attachment descriptors together with the message.
10.20	Apply direct-chat blocking/contact gates before delivery.

SOURCE REFERENCES

client/src/components/chat/MessageComposer.jsx
client/src/store/useChat.js
server/controllers/messageController.js

MODULE 11 / 16 ENTRIES

Message history and reading experience

Conversation participants

11.01	Load the newest page of message history.
11.02	Scroll back to load older history pages.
11.03	Use timestamp plus message ID for stable pagination.
11.04	Avoid duplicate messages when pages and realtime updates overlap.
11.05	Show date separators and message timestamps.
11.06	Distinguish your outgoing messages from received messages.
11.07	Render text, image, video, document and recorded-message bubbles.
11.08	Render location, live-location and poll messages.
11.09	Show sender identity in a group conversation.
11.10	Show system messages for group actions.
11.11	Show quoted reply context.
11.12	Jump to a quoted or searched message with a context window.
11.13	Return to the latest messages after viewing older context.
11.14	Keep newer incoming messages when a local delete request fails.
11.15	Show loading, empty-history and unavailable-message states.
11.16	Prevent outsiders from reading private conversation history.

SOURCE REFERENCES

client/src/components/chat/MessageList.jsx
client/src/components/chat/MessageBubble.jsx
server/controllers/messageController.js

MODULE 12 / 19 ENTRIES

Message actions, reactions and forwarding

Conversation participants; edits/retraction restricted to sender

12.01 Reply to a selected message.

12.02 Reject replies to deleted or inaccessible targets.

12.03 Copy supported message text to the clipboard.

12.04 Edit your own eligible message.

12.05 Limit editing to the existing five-minute edit window.

12.06 Display edited-message state.

12.07 Delete a message for yourself.

12.08 Delete your own eligible message for everyone.

12.09 Limit delete-for-everyone to the existing five-minute window.

12.10 Show a removed-message placeholder after retraction.

12.11 Add an emoji reaction to a message.

12.12 Change or remove your reaction through the reaction action.

12.13 Display reaction counts and updates.

12.14 Forward a message to selected chats.

12.15 Search destination chats in the forward picker.

12.16 Preserve forwarded attribution metadata.

12.17 Forward supported attachments, including verified video notes.

12.18 Reject edits or retractions attempted by another sender.

12.19 Create a team task from an accessible team message.

SOURCE REFERENCES

client/src/components/chat/MessageBubble.jsx
client/src/components/modals/ModalHost.jsx
server/controllers/messageController.js

MODULE 13 / 10 ENTRIES

Starred messages

Signed-in users; stars belong to the individual user

13.01 Star a message from its action menu.

13.02 Unstar a previously starred message.

13.03 Open the dedicated Starred page.

13.04 List starred messages across accessible conversations.

13.05 Show source conversation and sender context.

13.06 Open a starred message in its conversation.

13.07 Remove a star directly from the starred list.

13.08 Keep stars separate from chat-wide temporary pins.

13.09 Respect membership and personal-deletion visibility.

13.10 Show an empty state when no starred messages are available.

SOURCE REFERENCES

client/src/pages/StarredPage.jsx
server/controllers/messageController.js

MODULE 14 / 15 ENTRIES

Temporary pinned messages

Direct-chat participants; group owners/admins can pin

14.01 Pin an eligible message for all conversation participants.

14.02 Choose a pin lifetime of 1 hour.

14.03 Choose a pin lifetime of 6 hours.

14.04 Choose a pin lifetime of 12 hours.

14.05 Choose a pin lifetime of 24 hours.

14.06 Display active pins in the conversation banner.

14.07 Open or jump to a pinned message.

14.08 Change/extend an existing pin through the pin operation.

14.09 Unpin a message when permitted.

14.10 Keep a maximum of three active pins per conversation.

14.11 Evict the oldest pin when a new pin exceeds the cap.

14.12 Expire pins after their chosen duration.

14.13 Restrict group pin actions to the permitted group roles.

14.14 Update pins through realtime events.

14.15 Load active pins with the first message-history page.

SOURCE REFERENCES

server/utils/pins.js
client/src/components/chat/PinnedBanner.jsx
server/controllers/messageController.js

MODULE 15 / 16 ENTRIES

Typing, presence and delivery receipts

Privacy and tenant capabilities apply

15.01	Show another participant typing in an open chat.
15.02	Stop the typing indicator when typing ends.
15.03	Show online/offline presence where permitted.
15.04	Show last-seen information where permitted.
15.05	Track presence across multiple sockets/devices.
15.06	Send periodic presence pings.
15.07	Show a single tick for sent messages.
15.08	Show delivered acknowledgement state.
15.09	Show read acknowledgement state.
15.10	Mark a conversation read when the recipient opens/reads it.
15.11	Update unread counts after read acknowledgements.
15.12	Honor read-receipt privacy on sending and displaying read states.
15.13	Deduplicate repeated client read receipts for the same message.
15.14	Reset receipt deduplication appropriately after reconnect.
15.15	Require membership before joining private chat socket rooms.
15.16	Refresh conversation state after realtime changes and reconnects.

SOURCE REFERENCES

server/socket/index.js
client/src/hooks/useSocket.js
client/src/lib/chatReadReceipts.js
server/utils/presence.js

MODULE 16 / 23 ENTRIES

Photos, videos, documents and upload limits

Members with attachment access; local or configured Cloudinary storage

16.01 Select and send images from the photo picker.

16.02 Send ordinary video files through the supported upload flow.

16.03 Send supported documents and file attachments.

16.04 Upload multiple files in one request, up to 10 files.

16.05 Limit each image to 10 MB.

16.06 Limit each ordinary video or file to 50 MB.

16.07 Limit video notes to 10 MB.

16.08 Validate file extension/type against the upload allow-list.

16.09 Support JPEG, JPG, PNG, GIF and WebP image extensions.

16.10 Support MP4, WebM and MOV video extensions.

16.11 Support MP3, WAV, OGG and M4A audio extensions.

16.12 Support PDF, DOC/DOCX, XLS/XLSX, PPT/PPTX, ZIP and TXT files.

16.13 Show readable errors for unsupported types, too many files and oversized uploads.

16.14 Check stricter image limits after multipart processing.

16.15 Clean up rejected local image uploads.

16.16 Return attachment URL, name, size and MIME metadata.

16.17 Store uploaded bytes on local disk or the configured cloud driver.

16.18 Issue a short-lived access token for protected local media URLs.

16.19 Verify actual local video-note bytes at message send and schedule time.

16.20 Verify signed cloud upload size metadata without additional database reads.

16.21 Verify legacy owned cloud clips through the configured provider when needed.

16.22 Preserve upload proofs through message forwarding and scheduling.

16.23 Keep unverified external video-note URLs from bypassing the upload limit.

SOURCE REFERENCES

server/middleware/upload.js
server/controllers/uploadController.js
server/utils/uploadMetadata.js
client/src/lib/api.js

MODULE 17 / 19 ENTRIES

Camera, voice notes and round video notes

Browser device permission and recording support required

17.01 Open the in-chat camera capture interface.

17.02 Preview the live camera image before capture.

17.03 Switch front/back cameras when supported by the device.

17.04 Capture a photo and send it to the conversation.

17.05 Close camera capture and release the media stream.

17.06 Record an audio voice note using the microphone.

17.07 Show elapsed voice recording time.

17.08 Send the recorded voice note.

17.09 Cancel and discard a voice recording.

17.10 Record a round video note from the composer.

17.11 Show video-note recording time and preview.

17.12 Stop video-note recording at 60 seconds.

17.13 Send or discard the video-note recording.

17.14 Reject a video note larger than 10 MB before upload.

17.15 Use the dedicated stricter video-note upload endpoint option.

17.16 Render video notes with a distinct circular presentation.

17.17 Play received voice/audio and video messages with supported browser controls.

17.18 Show permission/device-unavailable errors when capture cannot start.

17.19 Release camera/microphone tracks when the capture UI closes.

SOURCE REFERENCES

client/src/components/chat/MessageComposer.jsx
client/src/components/chat/MessageBubble.jsx

MODULE 18 / 8 ENTRIES

View-once media

Image/video message participants

18.01	Mark an eligible image/video message as view-once.
18.02	Show the view-once state instead of an ordinary media preview.
18.03	Open view-once media through the viewed operation.
18.04	Record which recipient opened the item.
18.05	Remove consumed media attachment data from the message after viewing.
18.06	Prevent normal repeat opening once the item is consumed.
18.07	Keep view-once behavior separate from disappearing-message timers.
18.08	Restrict the view-once flag to supported image/video message types.

SOURCE REFERENCES

server/controllers/messageController.js
client/src/components/chat/MessageComposer.jsx
client/src/components/chat/MessageBubble.jsx

MODULE 19 / 12 ENTRIES

Chat polls

Members who may post in the conversation

19.01 Open the Create poll composer.

19.02 Enter a poll question.

19.03 Add and edit answer options.

19.04 Remove an answer option before sending.

19.05 Choose single-answer or multiple-answer voting.

19.06 Send the poll as a conversation message.

19.07 Vote for a permitted choice or choices.

19.08 Update an existing vote through the poll vote flow.

19.09 Show vote totals and option result bars.

19.10 Update poll votes through realtime events.

19.11 Validate option indexes and poll contents on the server.

19.12 Apply membership, group posting and direct-chat blocking rules.

SOURCE REFERENCES

client/src/components/chat/PollCard.jsx
client/src/components/chat/MessageComposer.jsx
server/controllers/messageController.js

MODULE 20 / 18 ENTRIES

Scheduled messages

Conversation members; dispatcher runs on the server

20.01	Open the Schedule message dialog.
20.02	Write the text to send later.
20.03	Select a custom date and time.
20.04	Choose the one-hour scheduling shortcut.
20.05	Choose the three-hour scheduling shortcut.
20.06	Choose tonight at 8 PM when that time is still available.
20.07	Choose tomorrow at 9 AM.
20.08	Create a scheduled text/media message through the API.
20.09	Require at least 10 seconds of scheduling lead time.
20.10	Limit one sender to 50 pending scheduled messages per chat.
20.11	List your own pending/failed scheduled items in a conversation.
20.12	Cancel your own scheduled message.
20.13	Send automatically even when the sender's browser is offline.
20.14	Dispatch through the normal delivery and authorization flow.
20.15	Recheck reply visibility and video-note upload size.
20.16	Track pending, sent, cancelled or failed delivery state.
20.17	Link a successful scheduled row to the delivered message.
20.18	Notify the author's other devices when schedule state changes.

SOURCE REFERENCES

client/src/components/chat/MessageComposer.jsx
server/controllers/messageController.js
server/utils/scheduledDispatcher.js

MODULE 21 / 12 ENTRIES

Location and live location

Browser location permission; conversation membership

21.01 Share your current static geographic location.

21.02 Attach latitude, longitude and a location label to a message.

21.03 Render the location as a dedicated message card.

21.04 Start a live-location sharing session.

21.05 Choose the available live-location sharing duration.

21.06 Update the active location while browser tracking is running.

21.07 View active live-location shares for the conversation.

21.08 Receive live location updates in realtime.

21.09 Stop an active share manually.

21.10 Expire a live share after its selected lifetime.

21.11 Restrict location updates/stopping to the share owner.

21.12 Handle denied or unavailable browser geolocation.

SOURCE REFERENCES

client/src/components/chat/MessageComposer.jsx
server/controllers/liveLocationController.js

MODULE 22 / 22 ENTRIES

Groups, group members and group roles

Workspace/contact restrictions and group roles apply

22.01	Create a group with a name and selected members.
22.02	Add an optional group description.
22.03	View your group list and member counts.
22.04	Open a group conversation from the group list.
22.05	Edit the group name, description or avatar when permitted.
22.06	View group participants and their role labels.
22.07	Add eligible contacts/team members to the group.
22.08	Search contacts in the Add members picker.
22.09	Remove a group member when permitted.
22.10	Promote an eligible member to group admin.
22.11	Dismiss a group admin back to member when permitted.
22.12	Represent group owner, admin and member roles separately.
22.13	Restrict role changes/removal according to group permissions.
22.14	Set the group posting policy to all members or admins only.
22.15	Enforce the posting policy for normal and scheduled messages.
22.16	Share a group invitation link/QR code.
22.17	Join a group using its invitation code.
22.18	Honor the user's group-add permission setting.
22.19	Leave a group.
22.20	Handle owner departure with the group ownership rules.
22.21	Show group membership changes as system messages.
22.22	Start a group audio/video call from the conversation.

SOURCE REFERENCES

server/controllers/groupController.js
client/src/pages/GroupsPage.jsx
client/src/components/chat/RightPanel.jsx
server/utils/rbac.js

MODULE 23 / 12 ENTRIES

Communities and announcement groups

Signed-in users; community membership/role rules apply

23.01 Create a named community.

23.02 Add a community description.

23.03 Create the associated announcements group.

23.04 List communities available to the signed-in user.

23.05 Open community details and linked groups.

23.06 Join a community with its invite code.

23.07 Add an eligible existing group to a community.

23.08 Restrict group linking to permitted community administrators.

23.09 Use announcement-group posting restrictions.

23.10 Navigate from a community to its group chats.

23.11 Leave a community.

23.12 Apply community membership and access checks to detail operations.

SOURCE REFERENCES

server/controllers/communityController.js
client/src/pages/CommunitiesPage.jsx

MODULE 24 / 18 ENTRIES

Broadcast lists

Mutual contacts only; owner manages the list

24.01 Create a named broadcast list.

24.02 Select recipients from eligible mutual contacts.

24.03 Deduplicate recipient IDs and exclude the sender.

24.04 Support up to 256 recipients per list.

24.05 View your own broadcast lists and recipient counts.

24.06 Rename a broadcast list.

24.07 Edit its recipient list.

24.08 Delete your own broadcast list.

24.09 Send text through a broadcast list.

24.10 Send supported image/video/document broadcast attachments.

24.11 Deliver individually into each recipient's direct conversation.

24.12 Create a direct conversation for an eligible recipient when needed.

24.13 Recheck mutual contacts at send time.

24.14 Skip recipients blocked by the sender.

24.15 Skip recipients who have blocked the sender.

24.16 Return separate sent and skipped counts.

24.17 Emit incoming message/conversation updates to recipients.

24.18 Refresh affected chat-list caches after broadcast delivery.

SOURCE REFERENCES

server/controllers/broadcastController.js
client/src/pages/BroadcastsPage.jsx

MODULE 25 / 17 ENTRIES

Search and jump to message

Only permitted content; locked chats excluded globally

25.01 Open the global search interface from application navigation.

25.02 Search people, chats, messages and meetings together.

25.03 Group global search results by content type.

25.04 Debounce search input rather than request on every keystroke.

25.05 Require a useful minimum query length for global search.

25.06 Search permitted people by exact identifiers.

25.07 Search existing group chat names.

25.08 Search direct conversations associated with matching people.

25.09 Search message text only inside accessible chats.

25.10 Search meeting titles only among hosted/invited meetings.

25.11 Search within the currently open conversation.

25.12 Select a result and load its surrounding message context.

25.13 Highlight or focus the selected message in history.

25.14 Exclude locked chats from global search results.

25.15 Exclude personally deleted/unavailable messages from context lookup.

25.16 Cap result counts so responses remain bounded.

25.17 Escape user regex input and use indexed text search where available.

SOURCE REFERENCES

client/src/components/search/GlobalSearch.jsx
client/src/components/chat/ChatSearchResults.jsx
server/controllers/searchController.js
server/controllers/messageController.js

MODULE 26 / 25 ENTRIES

Status and 24-hour stories

Contacts and per-story audience rules apply

26.01 Create a text status update.

26.02 Choose the text-status background.

26.03 Create supported image/video status updates.

26.04 Attach a caption/content field where supported.

26.05 Show your own active status items.

26.06 Show contacts' status items grouped by person.

26.07 Open a full-screen status viewer.

26.08 Move between story items and people.

26.09 Automatically advance stories with the progress timer.

26.10 Hold/pause a story while viewing or typing a reply.

26.11 Use left/right keyboard arrows to navigate stories.

26.12 Close the story viewer with Escape.

26.13 Show story progress and seen/unseen state.

26.14 Expire status items after 24 hours.

26.15 Mark an allowed status as viewed.

26.16 Avoid counting the owner as their own viewer.

26.17 Avoid counting repeated views by the same viewer.

26.18 Show status viewers and view times to the owner.

26.19 Update owner view counts in realtime.

26.20 Reply to an allowed status.

26.21 Delete your own status update.

26.22 Limit story audience to contacts even for the broad audience option.

26.23 Support selected-contact and excluded-contact audiences through the API.

26.24 Keep viewer lists/reply-author data private from other viewers.

26.25 Refresh allowed contacts' feeds when a status is posted or deleted.

SOURCE REFERENCES

client/src/pages/StatusPage.jsx
server/controllers/statusController.js
server/models/Status.js

MODULE 27 / 32 ENTRIES

Direct and group audio/video calls

Browser media permission; contact/group and tenant feature gates

27.01	Start a direct audio call.
27.02	Start a direct video call.
27.03	Start a group audio call.
27.04	Start a group video call.
27.05	Show outgoing dialing/ringing state.
27.06	Show an incoming-call overlay with caller identity.
27.07	Accept an incoming call.
27.08	Reject an incoming call.
27.09	Cancel an outgoing call before connection.
27.10	End an active call.
27.11	Handle busy or already-in-call state.
27.12	Handle unanswered/missed calls.
27.13	Show call duration and connection state.
27.14	Mute or unmute your microphone.
27.15	Turn the camera off or on.
27.16	Share your screen when browser support allows it.
27.17	Restore the camera after screen sharing stops.
27.18	Toggle browser microphone noise cancellation.
27.19	Switch audio output/speaker where browser setSinkId support exists.
27.20	Show local self-preview and remote participant video tiles.
27.21	Minimize a call while continuing to use the application.
27.22	Restore/expand the minimized call.
27.23	Use mute/end controls from the minimized call bar.
27.24	Enter or exit full-screen call view.
27.25	Use responsive group-call tile layouts.
27.26	Switch between the available grid/stage call views.
27.27	Choose portrait or landscape video tile shape.
27.28	Open Add people and invite eligible contacts into the active call.
27.29	Introduce invited conference members to the existing call peers.
27.30	Close ringing overlays on your other devices after you handle a call.
27.31	Exchange offer/answer/ICE through authorized Socket.IO signaling.
27.32	Clean up media tracks and connections on hangup/disconnect.

SOURCE REFERENCES

server/controllers/callController.js
server/utils/callService.js
client/src/hooks/useWebRTC.js
client/src/components/overlays/CallOverlay.jsx

MODULE 28 / 17 ENTRIES

Call history and network relay support

Signed-in users; real relay operation requires provider configuration

28.01 List recent call records.

28.02 Show audio/video type, peer, time and duration.

28.03 Distinguish incoming, outgoing and missed call history.

28.04 Filter the call-history screen using its available call categories.

28.05 Call a person back from history.

28.06 Load authenticated ICE configuration before establishing media.

28.07 Use STUN to discover direct network connectivity.

28.08 Use configured self-hosted coturn relays first.

28.09 Use configured Metered relays as an additional provider.

28.10 Use configured Cloudflare relays as an additional provider.

28.11 Preserve server-side provider preference order.

28.12 Mint time-limited TURN credentials on the server.

28.13 Refresh credentials before the server-reported TTL expires.

28.14 Share one in-flight credential request across concurrent callers.

28.15 Return controlled STUN-only state when no relay is configured.

28.16 Show a useful relay/network explanation when connection fails.

28.17 Keep provider keys and relay shared secrets out of the frontend.

SOURCE REFERENCES

client/src/pages/CallsPage.jsx
server/controllers/rtpccred.js
server/utils/iceServers.js
client/src/lib/iceServers.js

MODULE 29 / 22 ENTRIES

Meeting scheduling and invitations

Signed-in users; host controls edits and invitations

29.01	Schedule a meeting with a title.
29.02	Store an optional meeting description.
29.03	Choose meeting date and start time.
29.04	Store meeting timezone and planned duration.
29.05	Choose audio or video meeting type.
29.06	Invite selected eligible users/contacts.
29.07	Invite additional people by email.
29.08	Associate a meeting with a chat through the API.
29.09	Generate a unique readable room code and shareable join link.
29.10	Copy the meeting ID or join link.
29.11	Invite additional people to an existing meeting as host.
29.12	RSVP Going, Maybe or Can't go.
29.13	Track pending RSVP responses.
29.14	View upcoming, past or all meetings.
29.15	Show meeting host, time, type and invitation state.
29.16	Edit permitted meeting details as host.
29.17	Cancel a meeting as host.
29.18	Send invitation/update notifications and configured email invitations.
29.19	Schedule a pre-meeting reminder using reminderMinutes.
29.20	Save the recurrence choice: none, daily, weekly or monthly.
29.21	Recurrence is stored only; automatic future occurrence generation is not implemented.
29.22	Retain scheduled, ongoing, completed and cancelled meeting states.

SOURCE REFERENCES

client/src/components/modals/ModalHost.jsx
client/src/pages/MeetingsPage.jsx
server/controllers/meetingController.js
server/utils/meetingReminders.js

MODULE 30 / 20 ENTRIES

Meeting join, password and device check

Signed-in attendee; host/admission policies apply

30.01 Join a meeting by code.

30.02 Join by a shareable meeting URL.

30.03 Open the meeting details before joining.

30.04 Preview the camera before entering the room.

30.05 Choose an available microphone.

30.06 Choose an available camera.

30.07 Test microphone input with the pre-join level indicator.

30.08 Play a speaker test tone.

30.09 Enable or disable pre-join noise suppression.

30.10 Report denied permissions or missing media devices.

30.11 Allow joining without a working device when the browser/client permits it.

30.12 Protect a meeting with an optional password.

30.13 Validate the password before issuing meeting access.

30.14 Wait for the host when join-anytime is disabled.

30.15 Knock/request admission for uninvited link joins when Ask to join is enabled.

30.16 Show pending admission and denied-entry states.

30.17 Let the host admit or deny a waiting participant.

30.18 Use a scoped, short-lived admission pass for the admitted attendee.

30.19 Apply mute-on-entry to non-host participants.

30.20 Reject cancelled, invalid, locked or unauthorized breakout rooms.

SOURCE REFERENCES

client/src/components/meeting/DeviceCheck.jsx
client/src/pages/MeetingRoom.jsx
server/controllers/meetingController.js
server/socket/index.js

MODULE 31 / 22 ENTRIES

Live meeting room and media controls

Admitted participants; browser and transport support apply

31.01	Show the meeting title, room ID and participant count.
31.02	Show your self-preview and remote participant tiles.
31.03	Mute or unmute your microphone.
31.04	Turn your camera on or off in video meetings.
31.05	Share a screen/window/tab where browser capture is supported.
31.06	Stop presenting and restore the camera stream.
31.07	Show a presenting banner and dedicated presenter stage.
31.08	Keep other participants in a strip while someone presents.
31.09	Switch video tile shape between portrait and landscape.
31.10	Enter or exit full-screen meeting mode.
31.11	Show connection/network quality.
31.12	Show media-permission and stream errors.
31.13	Warn when a peer-to-peer room exceeds comfortable capacity.
31.14	Use peer-to-peer mesh by default.
31.15	Use LiveKit SFU transport when configured on the server.
31.16	Adapt mesh encoding to the number of peer connections.
31.17	Record the meeting locally on the participant's device where supported.
31.18	Stop local recording and save/download the resulting local recording.
31.19	Auto-start local recording when the meeting's Auto-record option is enabled.
31.20	Show an active recording indicator.
31.21	Keep secondary controls in a mobile More options sheet.
31.22	Leave the room and clean up media/network resources.

SOURCE REFERENCES

client/src/pages/MeetingRoom.jsx
client/src/hooks/useMeetingRoom.js
client/src/hooks/useLiveKitRoom.js

MODULE 32 / 22 ENTRIES

Meeting chat, reactions, hands, polls and Q&A

Admitted participants; selected actions are host-only

32.01	Open or close the in-meeting chat drawer.
32.02	Send a text message to the live meeting room.
32.03	Show sender identity on meeting chat messages.
32.04	Show an unread meeting-chat badge while the drawer is closed.
32.05	Send a meeting emoji reaction.
32.06	Show live floating reactions.
32.07	Retain a bounded reaction history on the meeting.
32.08	Raise your hand.
32.09	Lower your own hand.
32.10	Show the queue of raised hands in order.
32.11	Allow the host to lower another participant's hand.
32.12	Create an in-meeting poll with question and answer options as permitted.
32.13	Support single-choice or multiple-choice meeting polls.
32.14	Vote and replace an existing vote.
32.15	Show live poll results.
32.16	Close a poll as host/permitted creator according to server rules.
32.17	Ask a Q&A question.
32.18	Ask anonymously through the supported Q&A option.
32.19	Upvote a question with per-user vote tracking.
32.20	Mark a question answered and attach an answer as host.
32.21	Persist meeting polls/questions separately from ordinary chat polls.
32.22	Restrict collaboration events to participants in the live room.

SOURCE REFERENCES

client/src/components/meeting/MeetingPollsPanel.jsx
client/src/pages/MeetingRoom.jsx
server/socket/index.js

MODULE 33 / 24 ENTRIES

Host controls, breakouts, notes and attendance

Host-only moderation/reporting; participants can use permitted notes

33.01	Lock or unlock a live meeting to additional entry.
33.02	Enable or disable participant meeting chat.
33.03	Enable or disable participant screen sharing.
33.04	Mute all participants as host.
33.05	Force-mute a selected participant.
33.06	Remove a participant from the live meeting.
33.07	Create 2 to 4 breakout rooms as host.
33.08	Assign meeting participants into breakout rooms.
33.09	Send breakout invitation events to assigned participants.
33.10	List current breakout rooms.
33.11	Open an assigned breakout room.
33.12	Return from a breakout room to the main meeting.
33.13	End current breakout rooms as host.
33.14	Reject unassigned users and nested breakout creation.
33.15	Open shared meeting notes.
33.16	Edit and save shared notes up to 20,000 characters.
33.17	Reload notes changed by another participant.
33.18	Use version/timestamp conflict detection rather than overwrite another edit.
33.19	Create a team task from accessible meeting notes.
33.20	Record the first actual room start and final end time.
33.21	Track attendee first-join, last-leave and accumulated attendance duration.
33.22	Retain attendance across a participant's rejoins.
33.23	View the attendance report as host.
33.24	Show meeting time, duration, attendee count and attendee records in the report.

SOURCE REFERENCES

server/controllers/meetingController.js
server/socket/index.js
client/src/components/meeting/MeetingNotesPanel.jsx
client/src/pages/MeetingRoom.jsx

MODULE 34 / 22 ENTRIES

Notifications, sounds and Web Push

Signed-in account; OS push needs browser permission and VAPID setup

34.01 Open the in-app notification bell/list.

34.02 Show unread notification counts.

34.03 Mark one notification read.

34.04 Mark all notifications read.

34.05 Open the related chat, contact request, call or meeting from a notification.

34.06 Receive contact request and acceptance notifications.

34.07 Receive direct/group message notifications.

34.08 Receive call and meeting notifications.

34.09 Save in-app notifications independently of OS-push preferences.

34.10 Enable or disable message notifications.

34.11 Enable or disable group notifications.

34.12 Enable or disable call notifications.

34.13 Enable or disable meeting notifications.

34.14 Enable or disable notification sound.

34.15 Request browser desktop/push notification permission.

34.16 Subscribe the current browser to Web Push.

34.17 Unsubscribe a browser push endpoint.

34.18 Expose the public VAPID key through authenticated API access.

34.19 Handle a service-worker push and its notification click.

34.20 Focus/open the relevant application page from a push.

34.21 Respect muted chats, notification preferences and Do not disturb for push.

34.22 Remove expired push subscriptions after provider expiry responses.

SOURCE REFERENCES

client/src/components/layout/TopBar.jsx
server/controllers/notificationController.js
server/controllers/pushController.js
server/utils/push.js
client/public/sw.js

MODULE 35 / 20 ENTRIES

Privacy, blocking and reporting

Own privacy settings; API enforcement varies by field

35.01	Choose last-seen visibility: Everyone, My contacts or Nobody.
35.02	Choose online-status visibility: Everyone, My contacts or Nobody.
35.03	Choose profile-photo visibility: Everyone, My contacts or Nobody.
35.04	Choose About visibility: Everyone, My contacts or Nobody.
35.05	Enable or disable reciprocal read receipts.
35.06	Choose who may add you to groups: Everyone or Contacts.
35.07	Store the account's status audience preference.
35.08	Store settings in the account rather than only local component state.
35.09	Block or unblock a person.
35.10	Exclude blocked people from applicable discovery results.
35.11	Reject contact requests blocked in either direction.
35.12	Reject direct message delivery blocked in either direction.
35.13	Skip blocked broadcast recipients.
35.14	Restrict direct call/contact signaling through the applicable server gates.
35.15	Report a user.
35.16	Report a group.
35.17	Submit message/status target reports through the report API.
35.18	Provide report reason and optional description.
35.19	Keep report submission confidential from the reported party.
35.20	Enforce conversation, workspace and tenant membership checks on private data.

SOURCE REFERENCES

client/src/pages/SettingsPage.jsx
server/utils/privacy.js
server/controllers/userController.js
server/controllers/reportController.js

MODULE 36 / 18 ENTRIES

Presence preferences and appearance

Signed-in users; personal appearance does not restyle another user

36.01 Set manual presence to Available.

36.02 Set manual presence to Away.

36.03 Set manual presence to Busy.

36.04 Set manual presence to Do not disturb.

36.05 Suppress push/desktop interruptions in Do not disturb while retaining the bell.

36.06 Choose Light theme.

36.07 Choose Dark theme.

36.08 Follow the System theme.

36.09 Choose Teal, Indigo, Violet, Cyan, Emerald, Rose or Amber accents.

36.10 Set a default chat wallpaper for the account.

36.11 Override the wallpaper for one conversation.

36.12 Reset a conversation to the default wallpaper.

36.13 Use Default, Mint, Paper, Ink, Aurora, Dusk or Ocean presets.

36.14 Use Dots, Grid, Waves or Doodle wallpaper patterns.

36.15 Apply each wallpaper's light/dark variant.

36.16 Store a per-chat bubble-accent override through the theme API.

36.17 Enable or disable Enter-to-send.

36.18 Keep personal chat themes synchronized through saved account settings.

SOURCE REFERENCES

client/src/pages/SettingsPage.jsx
client/src/lib/wallpapers.js
server/controllers/userController.js
client/src/components/chat/RightPanel.jsx

MODULE 37 / 19 ENTRIES

Workspace setup, members and roles

Team owner/admin/member; platform admin has separate authority

37.01 Create a team workspace through the explicit team signup path.

37.02 Place individual accounts in the shared Personal space.

37.03 View workspace identity, type and your current workspace role.

37.04 View the team member roster.

37.05 Rename/edit allowed workspace settings as owner/admin.

37.06 Copy/share a workspace invite code or link.

37.07 Rotate the workspace invite code as owner/admin.

37.08 Join invited users as team members.

37.09 Promote eligible team members to workspace admin.

37.10 Demote an eligible admin to member.

37.11 Suspend an eligible workspace member.

37.12 Reactivate an eligible suspended member.

37.13 Remove an eligible member from the workspace.

37.14 Transfer workspace ownership to an eligible member as owner.

37.15 Change the previous owner's role according to the transfer flow.

37.16 Protect owner/self operations from invalid role/status changes.

37.17 Separate workspace permissions from group permissions.

37.18 Keep cross-workspace direct chats separate from team-owned group data.

37.19 Restrict team tools from Personal accounts and limited guests.

SOURCE REFERENCES

server/controllers/workspaceController.js
client/src/pages/SettingsPage.jsx
server/utils/rbac.js
server/utils/workspaceService.js

MODULE 38 / 19 ENTRIES

Shared customer inbox

Customer initiates a thread; team members handle team replies

38.01 Open a team's customer-contact screen.

38.02 Start a customer thread explicitly, without copying private direct messages.

38.03 Post further customer messages into the existing team thread.

38.04 List the customer's own team inbox conversations.

38.05 List incoming customer threads for the team.

38.06 Filter team inbox threads by open, pending or closed status.

38.07 Open a thread with its customer and message history.

38.08 Reply to a customer as a team member.

38.09 Assign a thread to an active teammate.

38.10 Clear the assignee to leave a thread unassigned.

38.11 Change thread status to Open, Pending or Closed.

38.12 Reopen a thread when the customer follows up.

38.13 Add a team-only internal note.

38.14 Show note author information to permitted team staff.

38.15 Remove internal notes from customer GET and follow-up responses.

38.16 Keep duplicate-create retry responses customer-safe.

38.17 Broadcast inbox-update hints to the team and customer in realtime.

38.18 Prevent another workspace from opening the team's threads.

38.19 Cap thread/message/note content to bound stored history.

SOURCE REFERENCES

server/controllers/workspaceToolsController.js
client/src/pages/WorkspacePage.jsx
client/src/pages/TeamContactPage.jsx

MODULE 39 / 22 ENTRIES

Team tasks and workspace activity

Active non-guest team members; activity is owner/admin-only

39.01 Create a manual team task.

39.02 Enter task title and optional description.

39.03 Assign a task to an active teammate.

39.04 Leave a task unassigned.

39.05 Set a future due date/time.

39.06 Create a task from an accessible team-chat message.

39.07 Create a task from accessible meeting notes.

39.08 Retain the task's source type and source ID.

39.09 List team tasks with assignee and creator context.

39.10 Mark a task Done.

39.11 Reopen a completed task.

39.12 Edit task title or description through the API.

39.13 Change or clear the assignee.

39.14 Change or clear the due date.

39.15 Reset reminder state when the due date changes.

39.16 Send due-task reminders through the task reminder worker.

39.17 Receive task-update hints in realtime.

39.18 Validate source message/meeting access before linking the task.

39.19 View workspace activity events as owner/admin.

39.20 Show activity actor, action, target and time.

39.21 Log supported member, inbox, task and guest actions.

39.22 Reject activity reads by ordinary members and foreign workspaces.

SOURCE REFERENCES

server/controllers/workspaceToolsController.js
server/utils/workspaceTaskReminders.js
client/src/pages/WorkspacePage.jsx

MODULE 40 / 21 ENTRIES

Limited guests and guest portal

Workspace owner/admin invites; guest sees only selected groups

40.01 Select 1 to 20 team groups for a guest invitation.

40.02 Generate a single-use guest signup link.

40.03 Keep the signup invitation valid for seven days.

40.04 Choose guest access for 1, 7, 30 or 90 days in the UI.

40.05 Choose access until revoked.

40.06 Accept 1 to 90 days or no expiry through the invite API.

40.07 Store the invitation token as a hash.

40.08 Register the guest into the invited team and permitted groups.

40.09 Open a dedicated restricted Guest groups portal.

40.10 Switch between the groups shared with that guest.

40.11 Read permitted group text history in the guest portal.

40.12 Send text-only replies to permitted groups.

40.13 Prevent guest file uploads and non-text posting.

40.14 Hide restricted attachments from the guest portal presentation.

40.15 Show guest access-expiry information.

40.16 List team guests and their active/suspended state as manager.

40.17 Revoke guest access immediately as owner/admin.

40.18 Remove the revoked guest from shared group memberships.

40.19 Invalidate revoked guest tokens.

40.20 Expire guest access through the guest-expiry worker.

40.21 Allow the guest to log out.

SOURCE REFERENCES

server/controllers/workspaceToolsController.js
server/utils/workspaceGuestExpiry.js
server/middleware/auth.js
client/src/pages/GuestPortal.jsx

MODULE 41 / 19 ENTRIES

Business storefront and automatic replies

Team business tools; owner/admin manages configuration

41.01 Open the team Business tools screen.

41.02 Set the business category.

41.03 Set the business description.

41.04 Set displayed business hours.

41.05 Set the business address.

41.06 Set the website link.

41.07 Set the business contact email.

41.08 Display the stored verified-business badge when granted.

41.09 Restrict business profile edits to permitted team roles.

41.10 Enable or disable the automatic greeting.

41.11 Edit the greeting message text.

41.12 Send the greeting once for the customer's first eligible conversation message.

41.13 Enable or disable the away reply.

41.14 Edit the away message text.

41.15 Set away business start and end hours.

41.16 Send an away response outside the configured business-hours window.

41.17 Throttle repeated away replies with the existing cooldown.

41.18 Run auto-reply checks off the normal message request path.

41.19 Avoid repeated configuration reads using the auto-reply cache.

SOURCE REFERENCES

client/src/pages/BusinessPage.jsx
server/controllers/workspaceController.js
server/utils/autoReply.js

MODULE 42 / 16 ENTRIES

Product catalog and product sharing

Team business owners/admins manage products; permitted users browse/share

42.01 View your team's product catalog.

42.02 Browse a business workspace's catalog through the API.

42.03 Create a product.

42.04 Set product name and description.

42.05 Set product price and currency.

42.06 Set product image information supported by the product form/API.

42.07 Set product URL information supported by the API.

42.08 Set product availability/inStock through the catalog API.

42.09 Edit an existing product.

42.10 Delete an existing product.

42.11 Prevent ordinary unauthorized members from editing the catalog.

42.12 Open the product picker from a team chat composer.

42.13 Select a product to share into a conversation.

42.14 Send the product's details through the product-share endpoint.

42.15 Apply normal conversation membership and send authorization to product sharing.

42.16 Show an empty catalog state when no products exist.

SOURCE REFERENCES

server/controllers/catalogController.js
client/src/pages/BusinessPage.jsx
client/src/components/chat/MessageComposer.jsx

MODULE 43 / 15 ENTRIES

Conversation labels and quick replies

Team workspace business tools

43.01 List the team's conversation labels.

43.02 Create a label with a name and supported color.

43.03 Delete a label.

43.04 Apply a label to a permitted conversation.

43.05 Remove a label from a permitted conversation through its apply/toggle flow.

43.06 Fetch labels assigned to one conversation.

43.07 Show assigned labels in the conversation information panel.

43.08 Keep team labels scoped to the workspace.

43.09 List saved quick replies.

43.10 Create a quick reply with its shortcut and text.

43.11 Edit a quick reply.

43.12 Delete a quick reply.

43.13 Find/select quick replies from the composer integration.

43.14 Insert a saved response into the current message draft.

43.15 Keep quick-reply content reusable without sending it until the user sends.

SOURCE REFERENCES

server/controllers/agentController.js
client/src/pages/BusinessPage.jsx
client/src/components/chat/RightPanel.jsx
client/src/components/chat/MessageComposer.jsx

MODULE 44 / 21 ENTRIES

Platform apps and tenant administration

App owner or platform administrator

44.01 Create a named embedded platform app/tenant.

44.02 List apps owned by the caller; platform admins can see the broader app set.

44.03 Display and copy the app ID.

44.04 Display/copy a newly issued app secret in its permitted reveal flow.

44.05 Rotate the app secret.

44.06 Enable or disable an app.

44.07 Set permitted host-platform origins.

44.08 Validate allowed origins for embedding and token/bootstrap requests.

44.09 Configure the app seat limit.

44.10 Configure end-user token lifetime.

44.11 View app-specific statistics and usage counts.

44.12 Enable/disable Direct messaging, Groups, Voice calls or Video calls.

44.13 Enable/disable Meetings, Status or Presence.

44.14 Enable/disable Typing, Read receipts or Reactions.

44.15 Enable/disable Attachments, Voice notes or Web Push.

44.16 Enforce capability gates on the server for embedded tenants.

44.17 Isolate app users and tenant-owned messaging data.

44.18 Permanently delete an app through the explicit confirmation flow.

44.19 Require typing the app ID to confirm permanent deletion in the UI.

44.20 Remove the app's provisioned users/data through the deletion cleanup flow.

44.21 Show and respect deletion-in-progress state.

SOURCE REFERENCES

client/src/pages/PlatformPage.jsx
server/controllers/platformController.js
server/models/App.js

MODULE 45 / 26 ENTRIES

Developer integration, hosted embed and SDKs

Host backend keeps app secret; browser receives user-scoped token

45.01	Verify backend app credentials using the platform identity endpoint.
45.02	Provision a host-platform user using a stable external ID.
45.03	Upsert the same external user idempotently.
45.04	Update supported provisioned profile fields.
45.05	List provisioned users for the authenticated app.
45.06	Deactivate an external user through the app backend API.
45.07	Issue a short-lived token for one provisioned end user.
45.08	Use the Node SDK createUserSession helper to provision and mint a session.
45.09	Embed hosted ChatKonect without building a second chat UI.
45.10	Bootstrap public app configuration before user authentication.
45.11	Fetch authenticated embed session information.
45.12	Fetch authenticated embed ICE configuration.
45.13	Initialize the browser drop-in loader with app/host/user-token configuration.
45.14	Mount the React ChatKonect component in a host application.
45.15	Select initial navigation path.
45.16	Select Light, Dark or System embed theme.
45.17	Provide a token directly or through a getToken callback.
45.18	Renew the token based on its validated JWT expiry.
45.19	Change identityKey when the signed-in host user changes.
45.20	Navigate the embedded interface through its exposed handle.
45.21	Trigger a token refresh through the exposed handle.
45.22	Destroy/unmount the embedded interface cleanly.
45.23	Receive ready, error, configuration, logout and navigation callbacks.
45.24	Use React type definitions for component props and ref methods.
45.25	Validate postMessage sender/origin in the hosted integration bridge.
45.26	Use the supplied host-platform integration example as an implementation reference.

SOURCE REFERENCES

server/controllers/platformController.js
server/controllers/embedController.js
client/public/embed.js
packages/react/src/index.d.ts
packages/node/src/index.js
client/src/pages/DevelopersPage.jsx

MODULE 46 / 21 ENTRIES

API keys, public REST API and incoming webhooks

Platform admins manage API keys; webhook management checks group access

46.01 Create an API key with a name and supported scopes.

46.02 Reveal the newly created key through its creation response.

46.03 List owned API keys with safe metadata.

46.04 Revoke an API key.

46.05 Authenticate public API requests with X-API-Key.

46.06 Check key identity and granted scopes.

46.07 Read contacts and search users with contacts:read scope.

46.08 Read chats/history with chat:read scope.

46.09 Create direct chats and send messages with chat:write scope.

46.10 Initiate calls with calls:write scope.

46.11 Read meetings with meetings:read scope.

46.12 Create meetings with meetings:write scope.

46.13 Run public API operations as the key owner with normal data-access checks.

46.14 Apply public API rate limiting.

46.15 List owned/permitted incoming webhooks.

46.16 Create an incoming webhook for a permitted group.

46.17 Generate an unguessable webhook ingress URL.

46.18 Post a supported message into the target group using the ingress token.

46.19 Delete/revoke a webhook.

46.20 Rate-limit webhook ingress.

46.21 Keep app secrets, user tokens, API keys and webhook tokens as separate credentials.

SOURCE REFERENCES

server/routes/v1Routes.js
server/controllers/apiKeyController.js
server/controllers/webhookController.js
client/src/pages/DevelopersPage.jsx

MODULE 47 / 22 ENTRIES

Admin dashboard, moderation and account control

Platform super-admin; separate from workspace admin

47.01 Open the platform administrator dashboard.

47.02 Show total users.

47.03 Show current active-user count.

47.04 Show total groups.

47.05 Show total messages.

47.06 Show total calls.

47.07 Show open report count.

47.08 Show the last seven days of user growth.

47.09 Show the last seven days of message volume.

47.10 List users with safe profile/account information.

47.11 Search users by name, username or email.

47.12 Set an eligible user's account to Active.

47.13 Suspend an eligible user.

47.14 Ban an eligible user.

47.15 Invalidate old tokens after suspension or banning.

47.16 Prevent an admin from suspending/banning themselves.

47.17 Delete another user and associated account data.

47.18 Prevent self-deletion through the admin delete endpoint.

47.19 List reports with reporter and target context.

47.20 Set report state to Open, Reviewing, Resolved or Dismissed.

47.21 Record security audit events for moderation changes.

47.22 Bootstrap/repair the configured super-admin account at startup.

SOURCE REFERENCES

client/src/pages/AdminDashboard.jsx
server/controllers/adminController.js
server/utils/createAdmin.js
server/utils/deleteUserAccount.js

MODULE 48 / 17 ENTRIES

Application shell, responsive UI and installation assets

All users; native browser capabilities vary

48.01 Navigate through the desktop rail and mobile bottom navigation.

48.02 Use a desktop conversation-list, message and information-panel layout.

48.03 Use mobile conversation navigation and back controls.

48.04 Open/close the conversation information panel.

48.05 Use responsive modals and mobile control sheets.

48.06 Show loading skeletons while data is being requested.

48.07 Show empty states when a module has no records.

48.08 Show success/error toasts for user actions.

48.09 Recover to an error boundary screen after an uncaught UI error.

48.10 Use accessible labels on icon-only controls and inputs.

48.11 Respect mobile safe-area spacing for bottom controls.

48.12 Retain theme-aware navigation, forms and message surfaces.

48.13 Expose a web app manifest and application icons for browser installation support.

48.14 Register the service worker used for notification handling.

48.15 Run the optional mock/demo mode to explore supported UI without live backend data.

48.16 Keep protected pages behind signed-in account access.

48.17 Route limited guests to their dedicated portal.

SOURCE REFERENCES

client/src/App.jsx
client/src/components/layout/AppLayout.jsx
client/src/components/layout/NavRail.jsx
client/src/components/layout/MobileNav.jsx
client/src/components/ErrorBoundary.jsx
client/public/manifest.webmanifest

MODULE 49 / 22 ENTRIES

Backend operation, data efficiency and safeguards

Deployment/operator capabilities; provider configuration may be required

49.01	Expose a health endpoint with database, email readiness and build identity.
49.02	Report boot time and current service time.
49.03	Use authenticated HTTP and Socket.IO access gates.
49.04	Apply configured cross-origin and embedding-origin restrictions.
49.05	Validate identifiers and bounded message/upload inputs.
49.06	Use MongoDB indexes for common message/history and lookup paths.
49.07	Compute chat-list unread totals in a shared aggregation.
49.08	Use optional Redis caching for hot chat lists.
49.09	Invalidate chat-list caches when relevant data changes.
49.10	Coalesce identical concurrent chat-list reads without delaying later requests.
49.11	Separate locked/unlocked and per-user cache scopes.
49.12	Run notification/auto-reply jobs through the queue abstraction.
49.13	Use BullMQ/Redis when configured and the supported inline fallback otherwise.
49.14	Run the scheduled-message dispatcher.
49.15	Run meeting reminder checks.
49.16	Run task reminder checks.
49.17	Run guest-expiry checks.
49.18	Use multi-instance Socket.IO/Redis support when configured.
49.19	Limit auth, public API and webhook ingress traffic.
49.20	Keep secrets and private authentication fields out of client responses.
49.21	Keep local media access authorization separate from ordinary user sessions.
49.22	Use read-receipt deduplication and bounded result/page sizes to reduce repeat work.

SOURCE REFERENCES

server/server.js
server/utils/chatCache.js
server/utils/readCoalescer.js
server/utils/queue.js
server/utils/redis.js
server/routes/index.js

READ THIS WITH THE CATALOGUE

Scope and implementation notes

Features are a source inventory, not a fresh end-to-end test sign-off.

01 This catalogue describes features present in the local repository on 20 October 2026. It is not a claim that every feature has been exercised on real devices or deployed.

02 Role and access notes describe the relevant module. Individual actions still require their server checks; a visible button is not a grant of permission.

03 Meeting recurrence choices are saved, but future recurring instances are not automatically generated. Live captions/transcription and background replacement are not exposed as working features in the current meeting UI.

04 Meeting recording is local browser/device recording, not a server recording archive. Device recording, screen capture, output switching and media quality depend on browser and device support.

05 Video-note bytes are verified server-side. The local recorder stops at 60 seconds and declared durations above 60 seconds are rejected; local video is not independently decoded to measure actual duration.

06 Profile-photo/About/status audience preferences exist. Visibility must be assessed at each consuming surface; saving a preference alone is not proof of universal privacy enforcement.

07 Workspace free/pro/business plan values and app limits are product configuration. No payment gateway, checkout, subscription charging or full billing/invoicing module is implemented here.

08 Email/OS push, Cloudinary, TURN and LiveKit require their respective deployment configuration. This PDF contains no private environment values, passwords or API secrets.

DEVELOPER REFERENCE

REST operation inventory

205 mounted operations from 28 route mounts plus system/shared ICE routes. Paths use the /api prefix. This index lists the surface, not full request schemas.

System

GET /api/health

/v1/embed

GET /api/v1/embed/config

GET /api/v1/embed/ice

GET /api/v1/embed/session

/auth

POST /api/auth/email/send-code

POST /api/auth/email/verify-code

POST /api/auth/signup

POST /api/auth/verify-otp

POST /api/auth/resend-otp

POST /api/auth/login

POST /api/auth/logout

POST /api/auth/refresh

GET /api/auth/me

GET /api/auth/sessions

POST /api/auth/sessions/revoke-others

DELETE /api/auth/sessions/:id

POST /api/auth/forgot-password

POST /api/auth/reset-password/:token

PATCH /api/auth/change-password

POST /api/auth/two-step/enable

POST /api/auth/two-step/change

POST /api/auth/two-step/disable

POST /api/auth/two-step/verify

DEVELOPER REFERENCE

REST inventory / continued

/auth

POST /api/auth/two-step/forgot

POST /api/auth/two-step/reset

GET /api/auth/turn-credentials

/users

GET /api/users/search

PATCH /api/users/me

PATCH /api/users/me/privacy

PATCH /api/users/me/presence

PATCH /api/users/me/settings

GET /api/users/me/export

DELETE /api/users/me

GET /api/users/me/contacts

POST /api/users/me/contacts/:id

DELETE /api/users/me/contacts/:id

POST /api/users/me/favorites/:id

POST /api/users/me/block/:id

PUT /api/users/me/chats/:chatId/theme

POST /api/users/me/chats/:chatId/:action

GET /api/users/:id

/chats

GET /api/chats

POST /api/chats/locked

POST /api/chats/direct/:userId

POST /api/chats/:id/lock

POST /api/chats/:id/unlock

GET /api/chats/:id

PATCH /api/chats/:id/disappearing

DEVELOPER REFERENCE

REST inventory / continued

/chats

DELETE /api/chats/:id/clear

DELETE /api/chats/:id

/messages

POST /api/messages

POST /api/messages/poll

POST /api/messages/read

GET /api/messages/starred

POST /api/messages/schedule

GET /api/messages/scheduled/:chatId

DELETE /api/messages/scheduled/:id

GET /api/messages/:chatId

GET /api/messages/:chatId/search

GET /api/messages/:chatId/pins

GET /api/messages/:chatId/context/:messageId

PATCH /api/messages/:id

DELETE /api/messages/:id

POST /api/messages/:id/react

POST /api/messages/:id/star

POST /api/messages/:id/pin

DELETE /api/messages/:id/pin

POST /api/messages/:id/vote

POST /api/messages/:id/viewed

/groups

POST /api/groups

POST /api/groups/join/:inviteCode

PATCH /api/groups/:id

POST /api/groups/:id/members

DEVELOPER REFERENCE

REST inventory / continued

/groups

DELETE /api/groups/:id/members/:userId

PATCH /api/groups/:id/members/:userId/role

POST /api/groups/:id/leave

/calls

GET /api/calls

GET /api/calls/history

POST /api/calls/start

POST /api/calls/end

POST /api/calls/missed

POST /api/calls/reject

POST /api/calls

PATCH /api/calls/:id

/meetings

GET /api/meetings

POST /api/meetings

GET /api/meetings/code/:code

GET /api/meetings/code/:code/rtc

POST /api/meetings/code/:code/join

GET /api/meetings/:id/report

GET /api/meetings/:id/notes

PUT /api/meetings/:id/notes

POST /api/meetings/:id/breakouts

GET /api/meetings/:id/breakouts

DELETE /api/meetings/:id/breakouts

PATCH /api/meetings/:id

POST /api/meetings/:id/rsvp

POST /api/meetings/:id/invite

DEVELOPER REFERENCE

REST inventory / continued

/meetings

DELETE /api/meetings/:id

/status

GET /api/status

POST /api/status

POST /api/status/:id/view

POST /api/status/:id/reply

GET /api/status/:id/viewers

DELETE /api/status/:id

/notifications

GET /api/notifications

PATCH /api/notifications/read

PATCH /api/notifications/:id/read

/contacts

GET /api/contacts/requests

POST /api/contacts/request/:userId

PATCH /api/contacts/request/:id

/reports

POST /api/reports

/upload

POST /api/upload

GET /api/upload/access

/admin

GET /api/admin/stats

GET /api/admin/users

PATCH /api/admin/users/:id/status

DEVELOPER REFERENCE

REST inventory / continued

/admin

DELETE /api/admin/users/:id

GET /api/admin/reports

PATCH /api/admin/reports/:id

/workspaces

GET /api/workspaces/customer/inbox

POST /api/workspaces/:workspaceId/inbox

GET /api/workspaces/me/inbox

GET /api/workspaces/me/inbox/:id

POST /api/workspaces/me/inbox/:id/replies

PATCH /api/workspaces/me/inbox/:id

POST /api/workspaces/me/inbox/:id/notes

GET /api/workspaces/me/tasks

POST /api/workspaces/me/tasks

PATCH /api/workspaces/me/tasks/:id

GET /api/workspaces/me/activity

GET /api/workspaces/me/guests

POST /api/workspaces/me/guests/invites

POST /api/workspaces/me/guests/:id/revoke

GET /api/workspaces/me

PATCH /api/workspaces/me

POST /api/workspaces/me/invite/rotate

POST /api/workspaces/me/transfer

PATCH /api/workspaces/me/members/:userId/role

PATCH /api/workspaces/me/members/:userId/status

DELETE /api/workspaces/me/members/:userId

/push

GET /api/push/key

DEVELOPER REFERENCE

REST inventory / continued

/push

POST /api/push/subscribe

POST /api/push/unsubscribe

/communities

POST /api/communities

GET /api/communities

POST /api/communities/join/:inviteCode

GET /api/communities/:id

POST /api/communities/:id/groups

POST /api/communities/:id/leave

/catalog

GET /api/catalog/mine

POST /api/catalog

POST /api/catalog/:id/share

PATCH /api/catalog/:id

DELETE /api/catalog/:id

GET /api/catalog/:workspaceId

/agent

GET /api/agent/labels

POST /api/agent/labels

GET /api/agent/labels/chat/:chatId

POST /api/agent/labels/:id/apply

DELETE /api/agent/labels/:id

GET /api/agent/quick-replies

POST /api/agent/quick-replies

PATCH /api/agent/quick-replies/:id

DELETE /api/agent/quick-replies/:id

DEVELOPER REFERENCE

REST inventory / continued

/broadcasts

GET /api/broadcasts

POST /api/broadcasts

POST /api/broadcasts/:id/send

PATCH /api/broadcasts/:id

DELETE /api/broadcasts/:id

/live-location

POST /api/live-location/start

GET /api/live-location/:chatId/active

POST /api/live-location/:messageId/update

POST /api/live-location/:messageId/stop

/search

GET /api/search

/webhooks

GET /api/webhooks

POST /api/webhooks

DELETE /api/webhooks/:id

/hooks

POST /api/hooks/:token

/keys

GET /api/keys

POST /api/keys

DELETE /api/keys/:id

/apps

DELETE /api/apps/:id/permanent

GET /api/apps/:id/stats

DEVELOPER REFERENCE

REST inventory / continued

/apps

POST /api/apps/:id/rotate

GET /api/apps

POST /api/apps

PATCH /api/apps/:id

DELETE /api/apps/:id

/v1/platform

GET /api/v1/platform/whoami

POST /api/v1/platform/users

GET /api/v1/platform/users

DELETE /api/v1/platform/users/:externalId

POST /api/v1/platform/tokens

/v1

GET /api/v1/me

GET /api/v1/contacts

GET /api/v1/users/search

GET /api/v1/chats

POST /api/v1/chats/direct/:userId

GET /api/v1/messages/:chatId

POST /api/v1/messages

POST /api/v1/calls

GET /api/v1/meetings

POST /api/v1/meetings

Shared ICE

GET /api/v1/ice

DEVELOPER REFERENCE

Incoming realtime events

48 registered Socket.IO input names, including compatibility aliases. A registered name still requires its handler's access checks.

- accept-call
- call-missed
- call-user
- call:accept
- call:answer
- call:busy
- call:cancel
- call:end
- call:ice-candidate
- call:introduce
- call:invite
- call:offer
- call:reject
- call:screen
- disconnect
- end-call
- join-chat
- leave-chat
- meeting:admit
- meeting:chat
- meeting:force-mute
- meeting:hand
- meeting:join
- meeting:leave
- meeting:lower-hand
- meeting:mute-all
- meeting:poll-close

DEVELOPER REFERENCE

Realtime events / continued

meeting:poll-create

meeting:poll-vote

meeting:presenting

meeting:qa-answer

meeting:qa-ask

meeting:qa-upvote

meeting:reaction

meeting:remove

meeting:signal

message-reaction

message-read

message:delivered

message:read

presence:ping

register-user

reject-call

typing-start

typing-stop

webrtc-answer

webrtc-ice-candidate

webrtc-offer

SOURCE COVERAGE

How this catalogue was assembled

145 referenced source files. The companion JSON records per-file SHA-256 fingerprints for this edition.

Product inventory: client pages, chat/meeting components, stores and hooks were matched with backend controllers, models and utility rules.

REST inventory: all router mounts in server/routes/index.js were enumerated, including appRoutes chained route declarations and the separate public webhook ingress router.

Realtime inventory: server/socket/index.js literal socket.on registrations and onAll compatibility aliases were enumerated.

Every source-reference path is checked for existence during PDF generation. Features receive stable module-local numbers for review and developer discussion.

The companion Markdown provides editable text. The JSON provides the module hierarchy, route list, incoming events and source fingerprints.

Historical feature/audit documents were used for context, while current source and the 20 October audit fixes take precedence where older documents disagree.

QUICK REFERENCE

REST surface begins on page 54.

Incoming realtime event surface begins on page 63.

Upload limits: image 10 MB; ordinary video/file 50 MB; video note 10 MB.

Recorder duration: video notes stop at 60 seconds.

Protected capabilities: app tokens, sessions, workspaces, groups and scoped credentials remain distinct.